01 login

01 — Login paso a paso (Compose + MVVM + Retrofit + .env)

> Objetivo: construir un formulario de login funcional que llegue a lo definido en `docs/api.md`.
> Restricciones y contexto: ver `docs/caso.md` §3.3, §3.4, §5.1 (MVP académico, datos 100% ficticios, stack Kotlin + Compose + M3 + MVVM + Room/SQLite + Retrofit, API 26+, 48x48dp, claro/oscuro).

0. A dónde hay que llegar (base: `api.md`)

```
curl -X POST https://172-233-15-71.ip.linodeusercontent.com/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"juan_perez","password":"Password123"}'
# -> 200 {access_token, token_type: "Bearer", expires_in: 3600, user: {...}}
```

La app debe hacer exactamente ese `POST`:

- `BASE_URL = https://172-233-15-71.ip.linodeusercontent.com/`
- `POST api/auth/login`
- body: `{ "username": "", "password": "" }`
- respuesta 200: `access_token`, `token_type`, `expires_in`, `user`

Regla innegociable de esta guía:

- **La URL jamás queda en duro en el source code.** Viene desde `.env` → `BuildConfig.API_BASE_URL` → `AppConfig`.
- Usa solo datos ficticios. Ej: `juan_perez / Password123`.

Buenas prácticas que aplica esta guía:

1. MVVM: `Screen (Compose, tonta)` → `ViewModel (estado + eventos)` → `Repository` → `Retrofit`.
2. Estado unidireccional con `StateFlow + LoginUiState` sellado o data class inmutable.
3. Componente reutilizable `AuthCard(content: @Composable ColumnScope.() -> Unit)` — el formulario va como `children` (slot API).
4. Validación en ViewModel, no en el Composable.
5. Accesibilidad caso §3.3: áreas táctiles ≥48dp, contraste, modo claro/oscuro, feedback visual inmediato, lenguaje simple.

Paso 1 — Dependencias

Archivo: `gradle/libs.versions.toml`

Agrega versiones (valores sugeridos por Android Studio, oct-2026):

```
[versions]
# ... las que ya tienes ...
retrofit = "3.0.0"
okhttpLogging = "5.5.0"
gsonConverter = "3.0.0"
lifecycleViewmodelCompose = "2.11.0"
coroutines = "1.11.0"

[libraries]
# ... las que ya tienes ...
retrofit = { group = "com.squareup.retrofit2", name = "retrofit", version.ref =
 "retrofit" }
retrofit-gson = { group = "com.squareup.retrofit2", name = "converter-gson",
 version.ref = "gsonConverter" }
okhttp-logging = { group = "com.squareup.okhttp3", name = "logging-interceptor",
 version.ref = "okhttpLogging" }
lifecycle-viewmodel-compose = { group = "androidx.lifecycle", name =
 "lifecycle-viewmodel-compose", version.ref = "lifecycleViewmodelCompose" }
# Icons del campo clave. 'core' trae Icons.Filled base; 'extended' trae
 Visibility/VisibilityOff.
# Sin core: Unresolved reference 'icons'. Sin extended: Unresolved reference
 'Visibility'.
androidx-compose-material-icons-core = { group = "androidx.compose.material", name =
 "material-icons-core" }
androidx-compose-material-icons-extended = { group = "androidx.compose.material", name
 = "material-icons-extended" }
coroutines-core = { group = "org.jetbrains.kotlinx", name = "kotlinx-coroutines-core",
 version.ref = "coroutines" }
coroutines-android = { group = "org.jetbrains.kotlinx", name =
 "kotlinx-coroutines-android", version.ref = "coroutines" }
```

Archivo: `app/build.gradle.kts`

Cambio aplicado: la plantilla inicial no traía `buildConfig = true` (AGP 8+ lo trae desactivado por defecto, por eso `BuildConfig` no existía). Se agregó para exponer `API_BASE_URL` sin hardcodear. Sin este flag Gradle no genera `com.rojas.syncpass.BuildConfig` y `AppConfig` no compila.

```
android {
    // ...
    buildFeatures {
        compose = true
        buildConfig = true // agregado: no existía en la plantilla inicial
    }
}

dependencies {
    // ... las que ya tienes ...
    implementation(libs.retrofit)
    implementation(libs.retrofit.gson)
    implementation(libs.okhttp.logging)
    implementation(libs.lifecycle.viewmodel.compose)
    implementation(libs.androidx.compose.material.icons.core)
    implementation(libs.androidx.compose.material.icons.extended) //
 Visibility/VisibilityOff viven aquí, no en core
    implementation(libs.coroutines.core)
    implementation(libs.coroutines.android)
}
```

Sincroniza Gradle: `Sync Now`.

Por qué así: Retrofit + Gson es lo pedido en caso §5.1 (`Retrofit`). Logging solo en debug para no filtrar tokens. `lifecycle-viewmodel-compose` permite `viewModel()` en Compose sin Hilt (suficiente para MVP).

Paso 2 — Variables de entorno con `.env` (nada en duro)

2.1 Crea `.env` (no se commitea)

En la raíz del proyecto (`SyncPass/.env`):

```
API_BASE_URL=https://172-233-15-71.ip.linodeusercontent.com/
```

2.2 Crea `.env.example` (sí se commitea)

`SyncPass/.env.example`:

```
# Copiar a .env y completar. Nunca commitear .env con valores reales.
API_BASE_URL=https://TU_HOST_O_IP/
```

2.3 Ignora `.env`

Agrega a `SyncPass/.gitignore` (y verifica `app/.gitignore`):

```
.env
```

2.4 Expón `.env` como `BuildConfig.API_BASE_URL`

Edita `app/build.gradle.kts` — arriba del bloque `android { }`:

```
import java.util.Properties

val envFile = rootProject.file(".env")
val envProps = Properties()
if (envFile.exists()) {
    envFile.inputStream().use { envProps.load(it) }
}
// Prioridad: .env > variable de entorno del sistema > fallback solo para compilar
val apiBaseUrl: String =
    envProps.getProperty("API_BASE_URL")
        ?: System.getenv("API_BASE_URL")
        ?: "https://172-233-15-71.ip.linodeusercontent.com/"

android {
    namespace = "com.rojas.syncpass"
    // ...
    defaultConfig {
        // ...
        buildConfigField("String", "API_BASE_URL", "\"$apiBaseUrl\"")
    }
}
```

2.5 Accede desde código con un solo punto

Nuevo archivo: `app/src/main/java/com/rojas/syncpass/core/config/AppConfig.kt`

```
package com.rojas.syncpass.core.config

import com.rojas.syncpass.BuildConfig

object AppConfig {
    val apiBaseUrl: String = BuildConfig.API_BASE_URL.trim().let {
        if (it.endsWith("/")) it else "$it/"
    }
}
```

Buena práctica: todo el código usa `AppConfig.apiBaseUrl`. Si ves `"https://..."` o `"172-233..."` en un `.kt`, está mal. Busca con: `rg -n "172-233|https?://" app/src/main/java --glob '*.kt'`

Paso 3 — Estructura MVVM que vamos a crear

Crea estos paquetes/archivos (todo ficticio, sin datos reales):

```
com.rojas.syncpass/
  core/config/AppConfig.kt              // paso 2.5
  data/remote/
    AuthApiService.kt                   // POST api/auth/login
    AuthDtos.kt                         // LoginRequest / LoginResponse / UserDto
    RetrofitClient.kt                   // usa AppConfig.apiBaseUrl
  data/repository/
    AuthRepository.kt                   // expone Result<LoginResponse>
  ui/components/
    AuthCard.kt                         // Card reutilizable con slot content
  ui/login/
    LoginUiState.kt                     // estado inmutable
    LoginViewModel.kt                   // validación + llamada a repo
    LoginScreen.kt                      // formulario como children de AuthCard
```

Qué es cada cosa (tipo Kotlin + rol):

Archivo	Tipo Kotlin	Qué es en simple
`core/config/AppConfig.kt`	`object` (singleton)	Caja única de configuración. Expone `apiBaseUrl` desde `BuildConfig`. `object` = una sola instancia en toda la app, sin `new`. No es `class` porque no necesita estado por instancia.
`data/remote/AuthDtos.kt`	3x `data class`	Moldes del JSON: `LoginRequest` (lo que envías), `LoginResponse` (lo que recibes: `access_token...`), `UserDto` (usuario anidado). `data class` = solo datos + `copy()/equals()` gratis. No llevan lógica.
`data/remote/AuthApiService.kt`	`interface`	Contrato HTTP: declara `suspend fun login(@Body): LoginResponse` con `@POST`. `interface` = solo firma, sin código. Retrofit la implementa por ti con proxies.

Archivo	Tipo Kotlin	Qué es en simple
`data/remote/RetrofitClient.kt`	`object` (singleton)	Fábrica del cliente HTTP: construye `OkHttp + Retrofit` una vez (`by lazy`) con `AppConfig.apiBaseUrl`. `object` porque debe ser único y compartido.
`data/repository/AuthRepository.kt`	`class` normal	Portero de datos: recibe la `interface` API por constructor y expone `suspend fun login(): Result<>`. `class` (no `object`) para poder inyectar un fake en tests.
`ui/components/AuthCard.kt`	`@Composable fun` (no es clase)	Componente visual reutilizable: un `Card` M3 con título + hueco `content` (slot). Función, no clase: Compose se escribe con funciones que describen UI.
`ui/login/LoginUiState.kt`	`data class` inmutable	Foto del estado de la pantalla: textos, errores, `isLoading`, `isSuccess`. Una sola fuente de verdad que el ViewModel emite y la UI observa.
`ui/login/LoginViewModel.kt`	`class : ViewModel()`	Cerebro de la pantalla: guarda el `StateFlow`, valida y llama al repository en `viewModelScope`. `class` que hereda de `ViewModel` para sobrevivir a rotación.
`ui/login/LoginScreen.kt`	`@Composable fun` (no es clase)	Pantalla: observa el estado y dibuja campos + botón dentro de `AuthCard`. Sin lógica de negocio, solo eventos hacia el ViewModel.

Regla rápida: `data class` = datos, `interface` = contrato sin código, `object` = único global, `class` = instancia con lógica testeable, `@Composable fun` = pieza de UI.

Por qué elegimos cada tipo (criterio de decisión)

¿Por qué `class` **normal?** → `AuthRepository`**,** `LoginViewModel`**:** Porque necesitan instancias con dependencias y ciclo de vida. `AuthRepository(api)` recibe la API por constructor: así en tests pasas

un fake sin tocar producción. Si fuera `object`, quedaría amarrado a un singleton global imposible de reemplazar. `LoginViewModel` debe ser `class : ViewModel()` porque el sistema crea una instancia por pantalla y la conserva ante rotación (ver detalle abajo); un `object` viviría para siempre y filtraría estado entre pantallas, y una `data class` no puede heredar ni manejar corrutinas.

¿Por qué `data class`**?** → `LoginRequest/Response/UserDto`**,** `LoginUiState`**:** Porque son solo datos que se comparan, copian y serializan. `data class` te da gratis `equals/hashCode/toString/copy/componentN`. Eso permite `state.copy(username = ...)` sin mutar el original (inmutabilidad para UDF) y que Gson convierta JSON <-> objeto sin código manual. Una `class` normal te obligaría a escribir todo eso a mano; un `object` no sirve porque necesitas miles de valores distintos, no uno único.

¿Por qué `object` **(singleton)?** → `AppConfig`**,** `RetrofitClient`**:** Porque debe existir uno solo y sin parámetros. `AppConfig.apiBaseUrl` es configuración global: no tiene sentido crear `AppConfig()` dos veces. `RetrofitClient.authApi` es costoso de crear (pool de conexiones OkHttp); con `object + by lazy` se crea una vez y se comparte. Si fueran `class`, cada ViewModel crearía su propio Retrofit y agotarías sockets. Si fueran `data class`, no tendrían sentido: no son valores comparables.

¿Por qué `interface`**?** → `AuthApiService`**:** Porque es un contrato sin implementación que otro genera. Declaras *qué* existe (`@POST login`) sin decir *cómo* se ejecuta por HTTP. Retrofit crea la implementación con proxies en runtime. Si fuera `class`, tendrías que escribir todo el código HTTP a mano. Además la `interface` permite el truco clave de tests: `AuthRepository(apiFake : AuthApiService)` donde `apiFake` es un anónimo que devuelve datos ficticios sin red.

¿Por qué `fun @Composable` **y no clase?** → `AuthCard`**,** `LoginScreen`**:** Porque Compose es declarativo por funciones, no por objetos-vista como el viejo XML (`findViewById`). Una función anota *qué dibujar según el estado*; Compose la re-ejecuta (recompone) cuando el estado cambia. Una `class View` guardaría estado mutable interno y rompería UDF. Por eso el slot es un parámetro función `content: @Composable ColumnScope.() -> Unit`: pasar UI como argumento solo es natural con funciones.

> Qué significa 'inmutabilidad para UDF': UDF (Unidirectional Data Flow) dice que los datos solo bajan (ViewModel -> UI) y los eventos solo suben (UI -> ViewModel). Para que eso sea predecible, el estado (`LoginUiState`) es inmutable: ningún Composable lo modifica directo. Cuando pasa algo, el ViewModel crea una copia nueva con `state.copy(username = ...)` y la emite por `StateFlow`. Compose ve un objeto nuevo y recompone. Si el estado fuera mutable (`var` compartido), dos partes podrían cambiarlo a la vez, la UI no sabría qué cambió y tendrías carreras y recomposiciones perdidas. Inmutable = cada cambio es un objeto nuevo, trazable y seguro entre hilos.

En resumen para decidir siempre:

1. ¿Es UI? → `@Composable fun`.
2. ¿Es solo datos comparables/copiables? → `data class`.
3. ¿Es contrato que otro implementa? → `interface`.
4. ¿Debe haber uno solo global sin dependencias? → `object`.
5. ¿Necesita dependencias, herencia o ciclo de vida? → `class`.

> Qué significa 'el sistema crea una instancia por pantalla y la conserva ante rotación': cuando rotas el teléfono, Android destruye y recrea la `Activity` y la composición de Compose (por eso se pierde todo lo guardado en `remember`). Pero el `ViewModel` no vive dentro de la Activity: vive en un `ViewModelStore` aparte que el sistema conserva durante el cambio de configuración. Al recrearse, la nueva pantalla pide `viewModel()` y recibe la misma instancia con su `StateFlow` intacto (usuario ya escrito, loading, etc.). Solo se destruye cuando sales de verdad de la pantalla (finish/back definitivo), llamando a `onCleared()`. Por eso el estado va en ViewModel y no en la UI.

Manifest — `app/src/main/AndroidManifest.xml`:

```
<manifest ...>
    <uses-permission android:name="android.permission.INTERNET" />
    <application android:label="@string/app_name" ...>
```

`android:label="@string/app_name"`: es el nombre visible de la app (launcher, recientes, título por defecto). Apunta a `@string/app_name` (no texto en duro) para soportar traducción y que Play Store/launcher lo muestren bien. Va en y se hereda a las activities que no declaren el suyo propio. Cambio aplicado: se quitó `android:theme="@style/Theme.SyncPass"` del porque duplicaba el mismo tema ya declarado en . Android Studio lo marca como redundante: la activity hereda el tema de la aplicación si no declara uno propio. Menos duplicación = un solo lugar donde cambiar el tema.

Paso 4 — DTOs + API (contrato de `api.md`)

Archivo: `data/remote/AuthDtos.kt`

```
package com.rojas.syncpass.data.remote

data class LoginRequest(
    val username: String,
    val password: String
)

data class LoginResponse(
    val access_token: String,
    val token_type: String = "Bearer",
    val expires_in: Long = 3600,
    val user: UserDto? = null
)

data class UserDto(
    val id: Int? = null,
    val username: String? = null,
    val email: String? = null
)
```

Archivo: `data/remote/AuthApiService.kt`

```
package com.rojas.syncpass.data.remote

import retrofit2.http.Body
import retrofit2.http.POST

interface AuthApiService {
    @POST("api/auth/login")
    suspend fun login(@Body body: LoginRequest): LoginResponse
}
```

Archivo: `data/remote/RetrofitClient.kt`

```
package com.rojas.syncpass.data.remote

import com.rojas.syncpass.BuildConfig
import com.rojas.syncpass.core.config.AppConfig
import okhttp3.OkHttpClient
import okhttp3.logging.HttpLoggingInterceptor
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

object RetrofitClient {
    // by lazy = inicialización perezosa y thread-safe: el bloque NO se ejecuta al
 cargar el object,
    // sino la primera vez que alguien usa RetrofitClient.authApi. Así no pagas el
 costo de crear
    // OkHttp/Retrofit al abrir la app si nadie hace login, y las siguientes veces
 reutiliza la misma
    // instancia (singleton). Sin lazy, se crearía al inicio aunque no se use.
    val authApi: AuthApiService by lazy {
        // .apply = scope function: crea el objeto (HttpLoggingInterceptor()) y dentro
 del bloque
        // el receptor es ese objeto como 'this' IMPLÍCITO (no se escribe). Por eso ves
 solo
        // 'level = ...' y no 'this.level = ...' ni 'logging.level = ...': ambas formas
 valen,
        // pero Kotlin te deja omitir 'this' cuando no hay ambigüedad.
        // Equivale a: val logging = HttpLoggingInterceptor(); logging.level = ...;
 pero en una sola expresión
        // que retorna el mismo objeto ya configurado. Si escribieras 'this.level =
 ...' sería idéntico.
        val logging = HttpLoggingInterceptor().apply {
            level = if (BuildConfig.DEBUG)
                HttpLoggingInterceptor.Level.BODY
            else
                HttpLoggingInterceptor.Level.NONE
        }
```

```
        val client = OkHttpClient.Builder()
            .addInterceptor(logging)
            .build()

        Retrofit.Builder()
            .baseUrl(AppConfig.apiBaseUrl) // <- nunca hardcodeada
            .client(client)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(AuthApiService::class.java)
    }
}
```

Por qué van OkHttp + Retrofit juntos: son dos capas distintas. **Retrofit** es la cara declarativa: conviertes una `interface` con `@POST`/`@Body` en llamadas tipadas que devuelven `data class` (vía Gson), sin armar JSON ni hilos a mano. Pero Retrofit no sabe hacer HTTP real por sí solo: por debajo delega en **OkHttpClient**, que es el motor que abre conexiones, reusa sockets (connection pool), pone timeouts/reintentos y ejecuta **interceptors** (aquí el logging, mañana un interceptor que agregue `Authorization: Bearer `). Sin OkHttp explícito, Retrofit usaría uno por defecto sin logging ni tuning; sin Retrofit, tendrías que usar OkHttp crudo y parsear JSON a mano. Por eso: Retrofit = qué pides, OkHttp = cómo viaja.

Archivo: `data/repository/AuthRepository.kt`

```kotlin
package com.rojas.syncpass.data.repository

import com.rojas.syncpass.data.remote.AuthApiService
import com.rojas.syncpass.data.remote.LoginRequest
import com.rojas.syncpass.data.remote.LoginResponse

// runCatching = ejecuta el bloque y atrapa cualquier excepción devolviendo Result:
// - si api.login() ok -> Result.success(LoginResponse)
// - si falla (sin red, 401, timeout, JSON roto) -> Result.failure(excepción), sin
 try/catch explícito.
// El ViewModel luego hace result.onSuccess { } / .onFailure { } para pasar a isSuccess
 o loginError.
class AuthRepository(private val api: AuthApiService) {
    suspend fun login(username: String, password: String): Result<LoginResponse> =
        runCatching {
            api.login(LoginRequest(username.trim(), password))
        }
}
```

Paso 5 — `AuthCard`: Card reutilizable que recibe el formulario como children

Archivo: `ui/components/AuthCard.kt`

Requisito del caso §3.3: botón/área ≥48dp, legible con luz variable, claro/oscuro, feedback inmediato. Por eso `AuthCard` centraliza estilo y el formulario entra por slot.

```kotlin
package com.rojas.syncpass.ui.components

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.ColumnScope
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.rojas.syncpass.ui.theme.SyncPassTheme

/**
 * Card reutilizable para auth. El formulario se pasa como [content] (children).
 * No sabe nada de login: solo layout + estilo.
 *
 * @param title Título principal del card (obligatorio, ej: 'SyncPass').
 * @param modifier Modifier externo para que el padre controle tamaño/posición.
 *  Va primero entre los opcionales por convención Compose. Se encadena con
 *  fillMaxWidth() interno: el padre puede sumar padding, click, etc. sin romper el
 ancho total.
 * @param subtitle Texto secundario opcional (nullable + default null = si no lo pasas,
 no se dibuja).
 * @param content Slot del formulario hijo. Tipo '@Composable ColumnScope.() -> Unit'
 significa:
 *  1) '@Composable' = es UI, se recompone cuando cambia su estado.
 *  2) '() -> Unit' = es una función sin retorno, solo dibuja (como children de React).
```

```
     *  3) 'ColumnScope.()' = función de extensión con receptor ColumnScope: dentro del
     bloque
     *     puedes usar Modifier.weight(), align(), etc. como si estuvieras dentro del Column
     *     interno del Card. Sin ese receptor el hijo no podría repartir espacio ni
     alinearse.
     */
    @Composable
    // Convención Compose: 'modifier' es el primer parámetro opcional (después de los
     obligatorios).
    // Si lo pones después de otro opcional como 'subtitle', el linter avisa
    // 'Modifier parameter should be the first optional parameter'.
    fun AuthCard(
        title: String,
        modifier: Modifier = Modifier,
        subtitle: String? = null,
        content: @Composable ColumnScope.() -> Unit
    ) {
        // Card M3 = contenedor elevado con forma, color y sombra del tema (se adapta a
     claro/oscuro solo).
        Card(
            modifier = modifier.fillMaxWidth(), // respeta el modifier del padre + ocupa
     todo el ancho
            elevation = CardDefaults.cardElevation(defaultElevation = 4.dp), // sombra sutil
            colors = CardDefaults.cardColors(
                containerColor = MaterialTheme.colorScheme.surfaceContainerHigh // color
     M3, no hex en duro
            )
        ) {
            // Column interno = apila título, subtítulo y formulario con ritmo vertical.
            Column(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(20.dp), // respiro interno fijo para uso en terreno con una
     mano
                horizontalAlignment = Alignment.CenterHorizontally, // centra
```

```
        título/subtítulo
                verticalArrangement = Arrangement.spacedBy(12.dp) // espacio uniforme entre
        hijos
            ) {
                // Text título con estilo tipográfico del tema (headlineSmall), hereda
        color onSurface.
                Text(
                    text = title,
                    style = MaterialTheme.typography.headlineSmall
                )
                // Subtítulo solo si viene no-nulo y no-blanco: evita hueco vacío cuando no
        se usa.
                if (!subtitle.isNullOrBlank()) {
                    Text(
                        text = subtitle,
                        style = MaterialTheme.typography.bodyMedium,
                        color = MaterialTheme.colorScheme.onSurfaceVariant // gris legible
        en ambas apariencias
                    )
                }
                Spacer(Modifier.height(4.dp)) // aire extra entre encabezado y formulario
                content() // <- aquí se dibuja el formulario hijo dentro del ColumnScope
            }
        }
    }

    @Preview(showBackground = true)
    @Preview(showBackground = true, uiMode = android.content.res.Configuration
     .UI_MODE_NIGHT_YES)
    @Composable
    private fun AuthCardPreview() {
        SyncPassTheme {
            AuthCard(
                title = "SyncPass",
                subtitle = "Vista previa con contenido ficticio"
            ) {
                Text("Aquí va el formulario hijo")
            }
        }
    }
```

Patrón slot: `content: @Composable ColumnScope.() -> Unit` permite que `LoginScreen`, `RegisterScreen`, `ForgotPasswordScreen` reutilicen el mismo Card sin duplicar estilo.

Paso 6 — Estado + ViewModel (validación aquí, no en la UI)

Archivo: `ui/login/LoginUiState.kt`

```
    package com.rojas.syncpass.ui.login

    data class LoginUiState(
        val username: String = "",
        val password: String = "",
        val isPasswordVisible: Boolean = false,
        val isLoading: Boolean = false,
        val usernameError: String? = null,
        val passwordError: String? = null,
        val loginError: String? = null,
        val isSuccess: Boolean = false
    ) {
        val canSubmit: Boolean =
            username.isNotBlank() && password.length >= 6 && !isLoading
    }
```

Archivo: `ui/login/LoginViewModel.kt`

```
package com.rojas.syncpass.ui.login

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.rojas.syncpass.data.remote.RetrofitClient
import com.rojas.syncpass.data.repository.AuthRepository
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch

// LoginViewModel = dueño del estado. La UI observa, el ViewModel decide.
// Es 'class : ViewModel()' (no object/data class) para que el sistema cree UNA
 instancia por
// pantalla y la conserve ante rotación, con viewModelScope que se cancela solo al
 salir.
class LoginViewModel(
    // repository por constructor con default = permite pasar un fake en tests sin
 tocar producción.
    private val repository: AuthRepository = AuthRepository(RetrofitClient.authApi)
) : ViewModel() {

    // CON guion (_uiState) = PRIVADO y MUTABLE: solo el ViewModel puede escribir.
    // MutableStateFlow guarda el LoginUiState actual y notifica a cada update.
    private val _uiState = MutableStateFlow(LoginUiState())
    // SIN guion (uiState) = PÚBLICO y SOLO LECTURA: la UI observa pero no puede mutar
 directo.
    // asStateFlow() expone la vista inmutable. Patrón backing property: escritura
 adentro, lectura afuera.
    // Así se cumple UDF: eventos suben (funciones onX), estado baja (uiState).
    val uiState: StateFlow<LoginUiState> = _uiState.asStateFlow()

    // onUsernameChange = evento de cada tecla en el campo usuario.
    // Guarda el texto nuevo y LIMPIA usernameError/loginError para que el error
```

```
        desaparezca al corregir.
           // Por qué update+copy (ver nota abajo): crea un estado NUEVO inmutable en vez de
        mutar el actual.
           fun onUsernameChange(value: String) {
               _uiState.update { it.copy(username = value, usernameError = null, loginError =
        null) }
           }

           // onPasswordChange = idem para clave. Limpia passwordError/loginError al escribir
        de nuevo.
           fun onPasswordChange(value: String) {
               _uiState.update { it.copy(password = value, passwordError = null, loginError =
        null) }
           }

           // onTogglePasswordVisibility = invierte el flag con !it.isPasswordVisible (toma el
        valor actual 'it').
           // Solo toca ese campo; copy conserva todo lo demás.
           fun onTogglePasswordVisibility() {
               _uiState.update { it.copy(isPasswordVisible = !it.isPasswordVisible) }
           }

           // onLoginClick = valida y luego llama a red. Flujo: lee snapshot, valida, loading,
        request, success/error.
           fun onLoginClick() {
               val current = _uiState.value // foto actual para validar sin suscribirse
               val usernameError = if (current.username.isBlank()) "Ingresa tu usuario o
        correo" else null
               val passwordError = if (current.password.length < 6) "Mínimo 6 caracteres" else
        null
               if (usernameError != null || passwordError != null) {
                   // Hay errores de formulario: los publica y NO llama a red (return
        temprano).
                   _uiState.update { it.copy(usernameError = usernameError, passwordError =
        passwordError) }
```

```
                return
            }
            // Sin errores: marca loading para deshabilitar botón (canSubmit) y mostrar
        spinner.
            _uiState.update { it.copy(isLoading = true, loginError = null) }
            // viewModelScope.launch = corrutina atada al ViewModel: no bloquea UI y se
        cancela al salir.
            viewModelScope.launch {
                val result = repository.login(current.username, current.password)
                result
                    .onSuccess { response ->
                        // TODO paso siguiente: guardar response.access_token en
        DataStore/Room
                        _uiState.update { it.copy(isLoading = false, isSuccess = true) }
                    }
                    .onFailure { e ->
                        // Mensaje genérico al usuario (no expone 401/timeout ni datos
        sensibles).
                        _uiState.update {
                            it.copy(
                                isLoading = false,
                                loginError = "No se pudo iniciar sesión. Revisa tus datos o
        tu conexión."
                            )
                        }
                    }
            }
        }

        // onErrorShown = la Screen lo llama tras mostrar el Snackbar para limpiar
        loginError y no repetirlo.
        fun onErrorShown() {
            _uiState.update { it.copy(loginError = null) }
        }
    }
```

```
// Por qué SIEMPRE 'update { it.copy(...) }' y no mutar directo:
// 1) LoginUiState es inmutable (val): no puedes hacer 'state.username = x', no
 compila. Solo copy crea otro.
// 2) 'update { }' hace lectura+escritura ATÓMICA y thread-safe: 'it' es el valor más
 reciente,
//    evita carreras si dos corrutinas actualizan a la vez (vs '_uiState.value = ...'
 que puede pisarse).
// 3) Cada update emite un OBJETO NUEVO por StateFlow -> Compose detecta cambio y
 recompone.
//    Mutar por dentro no emitiría nada y la UI quedaría desactualizada.
// En corto: update = forma segura de publicar, copy = forma de crear el estado
 siguiente sin romper el actual.
```

No guardes el token en memoria ni en `SharedPreferences` sin cifrar en la versión final. Para este MVP basta con `isSuccess`; el almacenamiento seguro (DataStore + offline-first con Room) es el siguiente documento.

Paso 7 — `LoginScreen`**: formulario como hijo de** `AuthCard`

Archivo: `ui/login/LoginScreen.kt`

```
package com.rojas.syncpass.ui.login

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.text.KeyboardActions
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Visibility
import androidx.compose.material.icons.filled.VisibilityOff
import androidx.compose.material3.Button
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.SnackbarHost
import androidx.compose.material3.SnackbarHostState
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.remember
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.input.ImeAction
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.text.input.VisualTransformation
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
```

```
import androidx.lifecycle.viewmodel.compose.viewModel
import com.rojas.syncpass.ui.components.AuthCard
import com.rojas.syncpass.ui.theme.SyncPassTheme

// LoginScreen = wrapper con estado: conecta ViewModel con UI. Sin dibujar directo.
// @param viewModel se obtiene con viewModel() = el sistema crea UNA instancia por
 pantalla
// atada al ciclo de vida (sobrevive a rotación). Nunca construir con LoginViewModel()
 a mano:
// eso rompe el scope y en preview ni hay ViewModelStoreOwner (warning 'Constructing a
 view model').
// @param onLoginSuccess callback al padre (navegar a Home).
@Composable
fun LoginScreen(
    viewModel: LoginViewModel = viewModel(),
    onLoginSuccess: () -> Unit = {}
) {
    // collectAsState() = suscribe el StateFlow; cada emisión recompone con el estado
 nuevo.
    // 'by' delega para usar 'state.username' directo en vez de 'state.value.username'.
    val state by viewModel.uiState.collectAsState()
    LoginContent(
        state = state,
        onUsernameChange = viewModel::onUsernameChange,
        onPasswordChange = viewModel::onPasswordChange,
        onTogglePassword = viewModel::onTogglePasswordVisibility,
        onLoginClick = viewModel::onLoginClick,
        onErrorShown = viewModel::onErrorShown,
        onLoginSuccess = onLoginSuccess
    )
}

// LoginContent = UI tonta y testeable: solo dibuja según 'state', sin ViewModel ni red.
// Todos los eventos entran como lambdas. Es lo que usa el preview con datos ficticios,
// así ningún @Preview construye un ViewModel (eso dispara el warning de Compose).
```

```
@Composable
fun LoginContent(
    state: LoginUiState,
    onUsernameChange: (String) -> Unit,
    onPasswordChange: (String) -> Unit,
    onTogglePassword: () -> Unit,
    onLoginClick: () -> Unit,
    onErrorShown: () -> Unit,
    onLoginSuccess: () -> Unit = {}
) {
    // SnackbarHostState en remember = sobrevive a recomposiciones (no a rotación; el
 error vive en ViewModel).
    val snackbar = remember { SnackbarHostState() }

    // LaunchedEffect(key) = efecto que se relanza cuando cambia la key. Aquí muestra
 el Snackbar
    // una vez por cada loginError nuevo y luego avisa onErrorShown() para limpiarlo y
 no repetirlo.
    LaunchedEffect(state.loginError) {
        state.loginError?.let {
            snackbar.showSnackbar(it)
            onErrorShown()
        }
    }
    // Cuando isSuccess pasa a true, notifica una vez al padre para navegar. No navega
 la Content directo.
    LaunchedEffect(state.isSuccess) {
        if (state.isSuccess) onLoginSuccess()
    }

    // Column raíz = centra el Card en pantalla con margen exterior de 16dp.
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
```

```
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            AuthCard(
                title = "SyncPass",
                subtitle = "Inicia sesión para acreditar asistencia"
            ) {
                // --- Todo lo de abajo es el 'children' (content: ColumnScope) del Card ---
                // Gracias al receptor ColumnScope podríamos usar Modifier.weight()/align()
aquí si hiciera falta.

                // OutlinedTextField usuario: controlado (value viene del state,
onValueChange va al ViewModel).
                // singleLine + isError/supportingText = error bajo el campo.
ImeAction.Next = pasa al siguiente campo.
                OutlinedTextField(
                    value = state.username,
                    onValueChange = onUsernameChange,
                    label = { Text("Usuario o correo") }, // label flotante M3, no
placeholder en duro
                    singleLine = true,
                    isError = state.usernameError != null,
                    supportingText = { state.usernameError?.let { Text(it) } },
                    keyboardOptions = KeyboardOptions(
                        keyboardType = KeyboardType.Email, // teclado con @ para una mano
en terreno
                        imeAction = ImeAction.Next
                    ),
                    modifier = Modifier.fillMaxWidth() // fillMaxWidth dentro del
ColumnScope del Card
                )

                // OutlinedTextField contraseña: visualTransformation oculta/muestra según
isPasswordVisible.
                // trailingIcon = IconButton con ojo + contentDescription accesible (lector
```

```
        de pantalla).
                    // KeyboardActions(onDone) = tecla Done del teclado dispara login, igual
        que el botón.
                    OutlinedTextField(
                        value = state.password,
                        onValueChange = onPasswordChange,
                        label = { Text("Contraseña") },
                        singleLine = true,
                        isError = state.passwordError != null,
                        supportingText = { state.passwordError?.let { Text(it) } },
                        visualTransformation = if (state.isPasswordVisible)
                            VisualTransformation.None
                        else
                            PasswordVisualTransformation(), // bullets •
                        trailingIcon = {
                            IconButton(onClick = onTogglePassword) {
                                Icon(
                                    imageVector = if (state.isPasswordVisible)
                                        Icons.Filled.VisibilityOff
                                    else
                                        Icons.Filled.Visibility,
                                    contentDescription = if (state.isPasswordVisible)
                                        "Ocultar contraseña" else "Mostrar contraseña"
                                )
                            }
                        },
                        keyboardOptions = KeyboardOptions(
                            keyboardType = KeyboardType.Password,
                            imeAction = ImeAction.Done
                        ),
                        keyboardActions = KeyboardActions(
                            onDone = { onLoginClick() }
                        ),
                        modifier = Modifier.fillMaxWidth()
                    )
```

```
            // Button Ingresar: enabled = canSubmit (derivado en UiState) evita taps
    inválidos.
            // height(48.dp) = mínimo táctil caso §3.3/§5.1 para operar con una mano.
            // Si isLoading muestra CircularProgressIndicator en vez de texto (feedback
    inmediato).
            Button(
                onClick = onLoginClick,
                enabled = state.canSubmit,
                modifier = Modifier
                    .fillMaxWidth()
                    .height(48.dp)
            ) {
                if (state.isLoading) {
                    // Progress dentro del botón con color onPrimary para contraste
    sobre el primario M3.
                    CircularProgressIndicator(
                        color = MaterialTheme.colorScheme.onPrimary
                    )
                } else {
                    Text("Ingresar")
                }
            }
        }
        // SnackbarHost = punto donde LaunchedEffect muestra el snackbar de loginError
    (red/credenciales).
        SnackbarHost(hostState = snackbar)
    }
}

// Preview SIN ViewModel: usa LoginContent con estado ficticio. Construir
 LoginViewModel() aquí
// dispara 'Constructing a view model in a composable' porque el preview no tiene scope
 de lifecycle
// y el ViewModel quedaría sin ViewModelStore. Así se previsualiza igual sin ese
```

```
 warning.
@Preview(showBackground = true)
@Composable
private fun LoginContentPreview() {
    SyncPassTheme {
        LoginContent(
            state = LoginUiState(username = "juan_perez", password = "Password123"),
            onUsernameChange = {},
            onPasswordChange = {},
            onTogglePassword = {},
            onLoginClick = {},
            onErrorShown = {}
        )
    }
}
```

Paso 8 — Conecta en `MainActivity`

Reemplaza el `Greeting` de plantilla en `MainActivity.kt`:

```
package com.rojas.syncpass

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import com.rojas.syncpass.ui.login.LoginScreen
import com.rojas.syncpass.ui.theme.SyncPassTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            SyncPassTheme {
                LoginScreen(
                    onLoginSuccess = {
                        // TODO 02: navegar a Home / lista de eventos
                    }
                )
            }
        }
    }
}
```

Borra `Greeting` y `GreetingPreview` de la plantilla.

Paso 9 — Probar

1 Verifica que `.env` existe y `BuildConfig` se generó:

`Build > Rebuild Project` y luego `rg -n "API_BASE_URL" app/build/generated -g '*.java' | head`

1 Ejecuta en emulador/dispositivo con internet.
2 Prueba camino feliz con datos ficticios de `api.md`:

- usuario: `juan_perez`
- clave: `Password123`
- Esperado: botón muestra loading → `isSuccess = true` (luego navegará).

1 Prueba errores:

- campo vacío → mensaje bajo el campo.
- clave <6 → `Mínimo 6 caracteres`, botón deshabilitado.
- sin internet → Snackbar `No se pudo iniciar sesión...`.

1 Contrasta con curl: la app debe enviar el mismo body que `docs/api.md`.
2 Revisa modo claro/oscuro y rotación: el estado sobrevive porque vive en `ViewModel`, no en `remember`.

Comando útil para confirmar que no hardcodeaste la URL:

```
rg -n "172-233-15-71|linodeusercontent|BASE_URL\s*=" app/src/main --glob '*.kt'
# Esperado: solo aparece en AppConfig.kt vía BuildConfig, nunca literal en
 RetrofitClient u otros.
```

Checklist de aceptación (para dar por listo el 01)

- [ ] POST llega a `api/auth/login` y con 200 marca `isSuccess`.

- [ ] `AuthCard` reutilizable con `content: @Composable ColumnScope.() -> Unit`; `LoginScreen` le pasa el formulario como hijo.
- [ ] Ningún `.kt` contiene la URL literal; todo pasa por `.env` → `BuildConfig` → `AppConfig`. `.env` está en `.gitignore`, `.env.example` sí commiteado.
- [ ] Validación en ViewModel, estados `loading / error / success` con feedback visual inmediato.
- [ ] Botón y campos ≥48dp, funciona en claro/oscuro, lenguaje simple.
- [ ] Solo datos ficticios en código, previews, capturas y logs (caso §5.1).
- [ ] `INTERNET` declarado; sin crash sin conexión.

Siguiente (no parte de este doc)

- `02_home.md`: llevar el `access_token` a Home, decodificar el JWT y mostrar la sesión (navegación por estado + `HomeViewModelFactory`).
- `03_session.md` (planificado): guardar el token en DataStore, auto-login y logout real.
- Más adelante: lista de proyectos/eventos, caché Room + `is_synced`, batch sync (caso §3.4).

Anexo — Explicación de conceptos (por qué la guía está hecha así)

Esta sección no es código nuevo: es el "por qué" detrás de cada decisión, para que puedas defenderla en la evaluación.

1. MVVM (Model-View-ViewModel)

- **View (UI):** `LoginScreen.kt`. Solo dibuja y reenvía eventos (`onUsernameChange`, `onLoginClick`). No valida, no llama a red. Se dice que es "tonta" a propósito: así es testeable y sobrevive a rotación.
- **ViewModel:** `LoginViewModel.kt`. Guarda el estado, valida reglas de negocio y orquesta la llamada al repositorio. Sobrevive a cambios de configuración (rotar el teléfono) porque no lo crea Compose, lo crea el sistema.
- **Model:** `AuthRepository + Retrofit + DTOs`. Sabe cómo obtener/guardar datos, no sabe nada de pantallas.

Flujo en una sola dirección (UDF — Unidirectional Data Flow):

```
Evento UI -> ViewModel.update(state) -> StateFlow emite -> UI recompone
```

Si rompes esto (ej: llamar a Retrofit directo desde el `Button`), mezclas capas y no puedes testear ni reutilizar.

2. `UiState` *+* `StateFlow`

- `LoginUiState` es una `data class` inmutable con todo lo que la pantalla necesita: valores, errores, `isLoading`, `isSuccess`. Una sola fuente de verdad.
- `MutableStateFlow` en privado + `StateFlow` público en solo lectura: la UI observa pero no muta directamente. Solo el ViewModel hace `_uiState.update { it.copy(...) }`.
- `collectAsState()` convierte ese Flow en `State` de Compose: cada emisión dispara recomposición solo de lo que cambió.
- `canSubmit` derivado evita lógica duplicada en el botón.

Por qué no `var` sueltos con `remember`: se pierden al rotar y se desincronizan entre campos.

3. Slot API / `children` *en Compose (*`AuthCard`*)*

```
fun AuthCard(..., content: @Composable ColumnScope.() -> Unit)
```

- Es el equivalente a `children` en React/Web: el padre define marco (Card, padding, título, elevación, colores M3) y el hijo define contenido (formulario login, registro, recuperar clave).
- `ColumnScope.()` como receptor permite que el hijo use `Modifier.weight()`, `align()`, etc., como si estuviera dentro del `Column` del Card.
- Beneficio caso §3.3: centralizas accesibilidad (padding 20dp, spacing 12dp, `surfaceContainerHigh` que se adapta a claro/oscuro) en un solo lugar. Si mañana cambia el diseño, cambias `AuthCard`, no 5 pantallas.

Mal uso común: pasarle `username/password` al Card. No: el Card no debe saber nada de login.

4. Compose básico usado aquí

- `@Composable`: función que describe UI, no la crea imperativamente. Se re-ejecuta (recompone) cuando cambia su estado observado.
- `Modifier`: cadena de decoradores (tamaño, padding, click). Orden importa. `fillMaxWidth().height(48.dp)` garantiza área táctil mínima pedida en caso §5.1.
- `remember { SnackbarHostState() }`: memoria local de composición (sobrevive a recomposiciones, no a rotación — por eso el error vive en ViewModel, no en remember).
- `LaunchedEffect(key)`: lanza una corrutina atada al ciclo de la composición. Sirve para efectos

secundarios *suspendidos* (mostrar un Snackbar, navegar, llamar a `onErrorShown`) que NO deben ejecutarse durante la composición misma (la composición debe ser pura y sin efectos). Reglas: 1) se lanza al entrar en composición y se **cancela al salir** (si el Composable se va, la corrutina muere: sin fugas); 2) si cambia la **key**, se cancela el anterior y se relanza (por eso `LaunchedEffect(state.loginError)` muestra el Snackbar una sola vez por cada error nuevo y no en cada recomposición); 3) vive en el scope de composición, NO en `viewModelScope`: para red usa el ViewModel, para UI usa LaunchedEffect. Error típico: llamar `snackbar.showSnackbar()` directo en el cuerpo del Composable (se relanzaría en cada recomposición en loop).

- `@Preview`: render sin emulador. Ponemos preview claro + oscuro para validar caso §3.3 sin correr la app.
- `MaterialTheme.colorScheme / typography`: nunca colores hex en duro; así soportas modo oscuro gratis.

5. Retrofit + DTOs + Repository

- `AuthApiService`: interfaz con anotaciones (`@POST`, `@Body`). Retrofit la implementa por ti con proxies. `suspend` la hace compatible con corrutinas (no bloquea el hilo UI).
- `LoginRequest / LoginResponse / UserDto`: DTOs (Data Transfer Objects) que calcan el JSON de `api.md`. `snake_case` (`access_token`) se deja igual al backend para no configurar `@SerializedName`; en dominio interno ya usarías `camelCase`.
- `GsonConverterFactory`: convierte JSON <-> data class automáticamente.
- `OkHttp + HttpLoggingInterceptor`: capa HTTP real. Logging en `BODY` solo si `BuildConfig.DEBUG`, si no `NONE` para no filtrar `access_token` en release.
- `RetrofitClient` como `object` + `by lazy`: singleton perezoso. Para MVP basta; en app real usarías Hilt/Koin.
- `AuthRepository` + `runCatching -> Result`: aísla el origen de datos. El ViewModel no sabe si viene de Retrofit, Room o fake para tests. Mañana agregas caché offline sin tocar el ViewModel.

6. Corrutinas (`viewModelScope.launch`)

- `viewModelScope` lanza trabajo asíncrono atado a la vida del ViewModel: si el usuario sale, se cancela solo, no fuga memoria.

- `suspend fun login()` se ejecuta fuera del hilo principal; Retrofit-OkHttp ya lo hace en IO. La UI solo ve `isLoading=true -> false`.
- `Result.onSuccess / onFailure`: manejo explícito sin `try/catch` regado. El mensaje al usuario es genérico ("Revisa tus datos o tu conexión") para no exponer detalles técnicos ni datos sensibles.

7. `.env` → `BuildConfig` → `AppConfig` *(nada en duro)*

- `.env`: archivo local no versionado con secretos/entornos. Cada dev/CI tiene el suyo.
- `.env.example`: plantilla versionada sin valores reales, documenta qué variables existen.
- `buildConfigField("String","API_BASE_URL",...)`: Gradle lee `.env` en tiempo de compilación y genera `BuildConfig.API_BASE_URL` como constante tipada. No es visible en el `.kt`, no se hardcodea.
- `AppConfig`: único punto de acceso en código + normaliza trailing `/` (Retrofit exige base URL terminada en `/`). Si ves la IP literal fuera del `build.gradle.kts` o `.env`, es deuda técnica.
- Prioridad `.env > System.getenv > fallback`: permite compilar en CI sin `.env` (vía secrets del runner) y en local con `.env`.

Por qué no `local.properties`: sirve, pero `.env` es estándar multiplataforma y el caso pide explícitamente "por ejemplo desde un .env". El mecanismo es el mismo: archivo ignorado + generación en build.

8. Validación y UX según caso §3.3

- Validar en ViewModel (`username.isBlank()`, `password.length < 6`) y exponer `usernameError/passwordError` + `supportingText`: la UI solo muestra, no decide.
- `canSubmit` deshabilita el botón: feedback preventivo, no solo correctivo.
- `OutlinedTextField` con `KeyboardOptions` (`Email/Password`, `ImeAction.Next/Done`) + `KeyboardActions(onDone)`: flujo con una mano en terreno.
- Mostrar/ocultar clave con `PasswordVisualTransformation` + `contentDescription` accesible.
- `Snackbar` para error de red + `CircularProgressIndicator` en botón para `isLoading`: "retroalimentación visual inmediata ante cada acción" que pide el caso.
- 48dp de alto en botón: mínimo táctil §5.1 para operar con una mano y con luz variable.

9. Qué NO hace este doc (y por qué)

- No guarda el token: hacerlo en `SharedPreferences` sin cifrar violaría buenas prácticas. Va en `02` con DataStore cifrado + Room para offline-first (caso §3.4: `is_synced`, `timestamps`, batch sync).
- No usa Hilt/Navigation: innecesario para un MVP de una pantalla; agrega fricción sin aporte. Se introduce cuando haya 2+ destinos.
- No loguea `password` ni `access_token`: el interceptor en release está en `NONE` y los previews usan datos ficticios (caso §5.1: prohibición de datos reales).

Glosario rápido:

Término	En una frase
DTO	Molde del JSON que viaja por red, no es tu modelo de dominio.
Repository	Portero: la UI pide "login", él decide si va a red, caché o fake.
StateFlow	Tubo observable de estado; emite, la UI recompone.
Slot	Hueco `content` donde el padre recibe hijos sin conocerlos.

Término	En una frase
BuildConfig	Clase generada en compilación con constantes por variante (aquí, la URL).
UDF	Los datos bajan, los eventos suben; nunca al revés.

Anexo — Troubleshooting: `3 issues were found when checking AAR metadata`

Síntoma

Al sincronizar/compilar aparece:

```
Dependency 'androidx.core:core-ktx:1.19.1' requires ... compile against version 37 or
 later ...
:app is currently compiled against android-36.1.
Recommended action: Update this project to use a newer compileSdk of 37.
```

Afecta también a `androidx.core:core:1.19.1` y `androidx.lifecycle:lifecycle-runtime-compose-android:2.11.0`.

Qué pasó

- `app/build.gradle.kts` declara `compileSdk = release(36) { minorApiLevel = 1 }` → `android-36.1`.
- `gradle/libs.versions.toml` trae `coreKtx = 1.19.1` y `lifecycleRuntimeKtx = 2.11.0`, que en su AAR `metadata` exigen `compileSdk >= 37`.
- Gradle valida esto en el check de metadata y falla. No es error de código, es incompatibilidad librería vs `compileSdk`.
- Recordar: `compileSdk` (con qué compilas) ≠ `targetSdk` (comportamiento runtime) ≠ `minSdk` (quién puede instalar, aquí 28).

Cómo resolverlo (2 opciones)

Opción A — Aplicada: subir `compileSdk` a 37.0 (no 37.1):

1. Tools > SDK Manager > instalar `Android API 37` (Platform + Build-Tools).
2. En `app/build.gradle.kts` (estado final del proyecto):

```
compileSdk {
    version = release(37) {
        minorApiLevel = 0
    }
}
```

Equivale a `android-37.0`. Se deja explícito `minorApiLevel = 0` para no caer en `37.1`.

1. Dejar `targetSdk = 36` y `minSdk = 28` como están. Sync + Rebuild.

Por qué `37.0` y no `37.1`:

- `core-ktx 1.19.1` y `lifecycle 2.11.0` exigen >=37, con `37.0` basta.
- AGP `9.2.1` está probado hasta `37.0`. Con `37.1` (`minorApiLevel = 1`) sale el warning `suppressUnsupportedCompileSdk=37.1` y aún no hay AGP estable que lo soporte.
- `targetSdk` se deja en 36: subirlo a 37 cambia comportamiento en runtime y solo deja un warning de lint en modo compatibilidad, sin bloquear el MVP.

Troubleshooting: `Unresolved reference retrofit2`

Qué significa: el código (`AuthApiService.kt`, `RetrofitClient.kt`) usa `import retrofit2....`, pero Gradle no tiene Retrofit en el classpath, así que Kotlin no lo encuentra. No es error de código, es dependencia faltante.

Por qué pasa (caso real de este proyecto): las versiones y alias sí existen en `gradle/libs.versions.toml` (`retrofit = 3.0.0`, `retrofit-gson`, `okhttp-logging`, etc.), pero faltan los `implementation(...)` en el bloque `dependencies` de `app/build.gradle.kts`. Sin esa línea, el artefacto se declara pero nunca se descarga ni se pone en compilación.

Cómo resolver: en `app/build.gradle.kts` agregar dentro de `dependencies { }`:

```
    implementation(libs.retrofit)
    implementation(libs.retrofit.gson)
    implementation(libs.okhttp.logging)
    implementation(libs.lifecycle.viewmodel.compose)
    implementation(libs.coroutines.core)
    implementation(libs.coroutines.android)
```

Luego Sync + Rebuild. Verifica en orden: 1) `[versions]` tiene la versión, 2) `[libraries]` tiene el alias, 3) `dependencies` tiene el `implementation`. Si falta el paso 3, verás exactamente este unresolved.

Opción B — Bajar librerías compatibles con 36 (si no puedes instalar SDK 37):

En `gradle/libs.versions.toml`:

```
    coreKtx = "1.17.0"
    lifecycleRuntimeKtx = "2.9.2"
```

Luego Sync.

Fuente contrato: `docs/api.md`. *Contexto negocio/restricciones:* `docs/caso.md`.